# pg_project 설치 가이드

대상 : naturalTuna/pg_project (2026-11 기준: pg_tune, pgBackRest 포함)
OS : RHEL 계열 8/9 (Rocky, Alma), x86_64
작성일 : 2026-11-09
같이 보기 : 아키텍처_가이드.pdf (그림으로 보는 구성 / 장애 동작)
운영가이드.txt (설치 후 관리 / 장애 대응)
pgmon_가이드.txt (모니터링 상세)

목차


1. 구성 요소
2. 구성 요약 (그림은 아키텍처_가이드.pdf)
3. 사전 준비 (체크리스트)
4. [1단계] 패키지 다운로드 - 인터넷 서버
5. [2단계] 폐쇄망 반입
6. [3단계] single 설치
7. [3단계] HA 설치
8. 무인 설치 (응답 파일)
9. 설치 후 확인
10. 동작 테스트
11. 설치 문제 해결 (실서버에서 겪은 것 포함)
12. 다시 설치할 때 (정리 방법)


## 1. 구성 요소

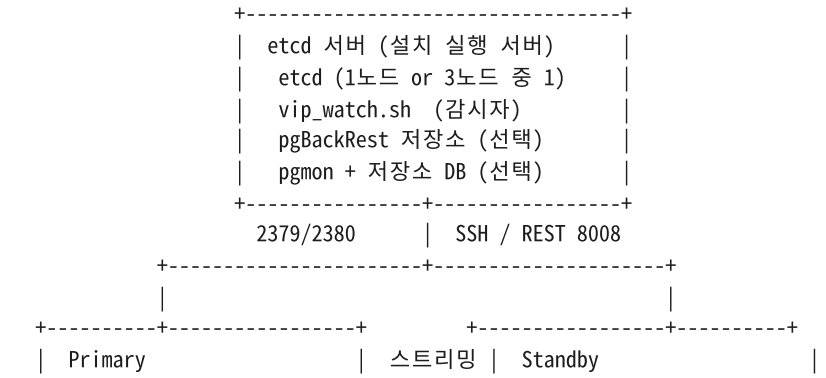
| | |
|---|---|
| pg_download.sh | [인터넷 서버] PG 소스, 의존 rpm, extension, pgBackRest, etcd, Patroni wheel, pgmon, pg_tune 을 postgresql/ 로 패키징 |
| postgresql_install.sh | [폐쇄망 서버] 소스 빌드 -> initdb -> 튜닝 -> 백업 -> (HA) Patroni |
| lib/common.sh | 출력, conf 생성, rpm 설치, PG/extension 빌드 |
| lib/input.sh | 입력 / 응답 파일 / 확인 화면 |
| lib/backup.sh | basebackup 백업, WAL 정리, PITR 복구 스크립트 |
| lib/pgbackrest.sh | pgBackRest 빌드 / 설정 / 크론 / 복구 스크립트 |
| lib/vip.sh | VIP (callback + 보정 cron + etcd 서버 감시자) |
| lib/ha_safety.sh | watchdog / failsafe_mode / synchronous_mode |
| lib/patroni_common.sh | Patroni 원격 설치 (install / add_node 공용) |
| lib/single.sh, ha.sh | 모드별 설치 |
| add_node.sh | HA 노드 추가 (A: 3-node / B: DR standby cluster) |
| ha_test.sh | HA 동작 자동 테스트 + 리포트 |
| monitoring/pgmon | 터미널 모니터링 (수집 데몬 + 알림), pgmon_check.sh 점검 |
| tools/pg_tune | 규칙 기반 쿼리 튜너 (hypopg / pg_hint_plan 검증) |

## 2. 구성 요약 (그림은 아키텍처_가이드.pdf)

### 2-1. single

DB 서버 1대: PostgreSQL + 백업(basebackup 또는 pgBackRest 로컬 저장소)
pgmon 저장소 DB 도 이 방식으로 설치 (HA 면 보통 etcd 서버에)

### 2-2. HA (3대)



```
|   PostgreSQL               | ------->  |   PostgreSQL               |
|   Patroni (REST 8008)      |   복제    |   Patroni (REST 8008)      |
|   etcd (3노드 선택 시)     |           |   etcd (3노드 선택 시)     |
|   vip_callback / vip_check |           |   vip_callback / vip_check |
|   softdog (watchdog)       |           |   softdog (watchdog)       |
|   [VIP] <- 앱 접속         |           |                            |
+----------------------------+           +----------------------------+
```

- etcd : 누가 Primary 인지(leader key) 저장. 3노드면 1대 장애에도 동작
- Patroni : PG 기동/정지/승격/복제 구성을 자동 관리
- VIP : Primary 를 따라 이동. 앱은 VIP 하나로 접속
- 장애 시 : Primary 다운 -> ttl 30초 -> Standby 승격 -> VIP 이동 (약 30~40초)

### 2-3. 포트 (방화벽 허용 필요)

| 출발 -> 도착 | 포트 | 용도 |
|---|---|---|
| 앱 -> VIP / Primary / Standby | 5432 | DB 접속 (입력한 포트) |
| Primary <-> Standby | 5432 | 복제, pg_rewind |
| DB 노드 -> etcd 노드 | 2379 | Patroni -> etcd |
| etcd 노드 <-> etcd 노드 | 2380 | etcd 피어 (3노드) |
| etcd 서버 -> DB 노드 | 8008 | Patroni REST (vip_watch, 설치 확인) |
| Primary <-> Standby | 8008 | Patroni REST (failsafe_mode) |
| etcd 서버 -> DB 노드 | 22 | SSH (설치, vip_watch, pgBackRest 백업) |
| DB 노드 -> etcd 서버 | 22 | SSH (pgBackRest archive-push) |
| Primary <-> Standby | 22 | SSH (설치) |
| pgmon 서버 -> DB 노드 | 5432 / 22 | 수집 / OS 지표 SSH fallback |

### 2-4. 설치 후 디렉토리 (기본값)

| | |
|---|---|
| /postgres/app | 엔진 (bin, lib, share) |
| /postgres/app/etc/configure_options.txt | 빌드 시 configure 옵션 |
| /postgres/app/etc/patroni | patroni.yml, vip_callback.sh, vip_check.sh (HA) |
| /postgres/app/etc/backup | basebackup/, wal_cleanup.sh, pitr_restore.sh |
| /postgres/app/etc/backup/pgbackrest | bin, lib(libpq), conf, log, pgbr_restore.sh |
| /pgdata/data | 데이터 |
| /pgdata/log | PG 로그, pg_install.log, ha_setup.log, vip.log |
| /pgbackup/arc, /pgbackup/back | basebackup 아카이브 / 백업 |
| /pgbackup/pgbackrest | pgBackRest 저장소 (single: 이 서버, HA: etcd 서버) |
| ~/.postgresrc | 환경변수 + alias (.bashrc 에서 로드) |
| ~/pgvip/vip_watch.sh | VIP 감시자 (etcd 서버) |
| /etc/etcd/etcd.conf | etcd 설정 (systemd: etcd) |
| /etc/systemd/system/patroni.service | Patroni 서비스 (DB 노드) |

## 3. 사전 준비 (체크리스트)

[공통]
[ ] 다운로드 서버 = 설치 서버와 같은 OS / 메이저 버전 (rpm, python 버전 일치)
[ ] 실행 계정 생성 (예: postgres) - 설치는 이 계정으로 실행
[ ] 실행 계정 sudo NOPASSWD (rpm 설치, systemd 등록, 디렉토리 생성)
    echo 'postgres ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/postgres
    chmod 440 /etc/sudoers.d/postgres
    (운영 시 축소 방법은 운영가이드 2-1)
[ ] 디스크: 엔진(/postgres), 데이터(/pgdata), 백업/아카이브(/pgbackup) 마운트
[ ] 최상위 디렉토리를 실행 계정 소유로 미리 생성
    sudo mkdir -p /postgres /pgdata /pgbackup
    sudo chown -R postgres:postgres /postgres /pgdata /pgbackup
    HA 는 etcd 서버 포함 3대 모두 (etcd 서버에서 빌드 + 로그 기록)
    pgBackRest(HA) 면 etcd 서버 /pgbackup 에 백업 들어갈 공간 확보
[ ] ko_KR.utf8 로케일 존재 확인 (locale -a | grep -i ko_KR)
[ ] crontab 사용 가능 (cronie) - 백업 / VIP / pgmon 크론

[HA 추가]
[ ] 세 서버 /etc/hosts 에 etcd / primary / standby 등록 (모든 서버 동일하게)
[ ] SSH 키 교환 (패스워드 없이, 실행 계정끼리)
    etcd 서버 -> primary, standby

```
      primary  <-> standby
      확인: ssh -o BatchMode=yes <호스트> hostname
      (pgBackRest 용 DB 노드 -> etcd 서버 키는 설치 스크립트가 자동 등록)
 [ ] 2-3 의 포트 방화벽 허용
 [ ] 시간 동기화 (chronyd) - etcd / Patroni 는 서버 간 시간이 맞아야 안정적
 [ ] (VIP 사용 시)
      같은 L2 서브넷의 미사용 IP 1개 확보  (ping 해서 응답 없어야 함)
      DB 노드에 iproute, iputils(arping), curl 설치 확인
      클라우드(AWS/GCP 등)는 ARP 기반 VIP 불가 -> LB/DNS 사용
 [ ] (watchdog 사용 시)
      DB 노드에 다른 watchdog 데몬(watchdog, wdmd 등)이 /dev/watchdog 를
      쓰고 있지 않은지:  ls -l /dev/watchdog*; sudo lsof /dev/watchdog
```

## 4. [1단계] 패키지 다운로드 - 인터넷 서버

```
$ git clone https://github.com/naturalTuna/pg_project.git
$ cd pg_project
$ bash pg_download.sh                          # 기본
$ bash pg_download.sh --verify                 # 시험 빌드까지 (빌드 환경 설치됨, 수 분 추가)
$ bash pg_download.sh --lock BUILD_LOCK.txt    # 예전과 같은 구성으로 다시 받기

질문 항목
  PostgreSQL 버전   지원 중인 버전 표에서 선택 (기본: 두 번째 최신 메이저의 최신 마이너)
  설치 모드         single / ha  (ha 면 etcd, Patroni wheel 추가)
  pgmon 포함        Y 권장
  의존 rpm 다운로드 Y  (CRB/PowerTools 자동 활성화, sudo 필요)
  pg_tune / pgBackRest   항상 포함

자동으로 정하는 것
  빌드 기능    받은 PG 소스의 configure 를 읽어 지원하는 옵션만 켬
               (lz4, zstd, uuid, systemd, io_uring=PG18+) -> 필요한 -devel 만 받음
               결과: installer/build_features.txt
  extension    GitHub 최신 정식 릴리스 태그 (hypopg, pg_hint_plan, pgvector)
               pg_hint_plan 은 PG 메이저 전용 태그(REL18_x_y_z) 를 자동 선택
  pgBackRest   최신 release/x.y.z 소스 + meson/ninja 휠 (rpm 못 받을 때 대비)
  rpm          manifest.json + 기능별 -devel, 의존성까지. 빠진 rpm 은 하나씩 재시도
               gcc / make / bison / flex 를 끝내 못 받으면 여기서 중단

--verify (선택)
  다운로드 서버에 빌드 환경을 설치하고 PG + extension + pgBackRest 시험 빌드
  extension 빌드가 실패하면 이전 태그로 자동 후퇴, 결과는 VERIFY_REPORT.txt

결과 확인
  postgresql/BUILD_LOCK.txt          받은 구성 기록 (PG, 기능, extension 태그, pgBackRest)
  postgresql/PKG/                    rpm 들  ->  ls postgresql/PKG | wc -l  (수백 개가 정상)
  postgresql/installer/              PG 소스, extensions/, pgbackrest/, (ha) etcd, patroni_pkgs
  postgresql/VERIFY_REPORT.txt       (--verify 시)

압축
$ tar -czf postgresql.tar.gz postgresql/
```

## 5. [2단계] 폐쇄망 반입

```
single : DB 서버로 복사
HA     : etcd 서버(설치 실행 서버)로만 복사. DB 노드는 설치 스크립트가 배포
         (DB 노드에 postgresql.tar.gz 를 따로 둘 필요 없음)

$ tar -xzf postgresql.tar.gz
$ cd postgresql
```

## 6. [3단계] single 설치

```
$ bash postgresql_install.sh
  설치 모드: 1) single

입력 항목 (괄호는 기본값)
  CPU core 수 / Memory(GB)   (현재 서버 값)   -> postgresql.conf 튜닝 기준
  PostgreSQL Port            (5432)
  엔진 설치 경로              (/postgres)      -> /postgres/app
  Data 영역 경로              (/pgdata)        -> /pgdata/data, /pgdata/log
  DB Locale                  (ko_KR.utf8)     -> 나중에 변경 불가 (재구축 필요)
  백업 선택                   (basebackup)     pgbackrest / basebackup
  [basebackup]
  Archive 사용               (Y)  상위 경로 (/pgbackup) -> /pgbackup/arc
  Backup 사용                (Y)  상위 경로 (/pgbackup) -> /pgbackup/back
    백업 주기                1) 매일 02:00  2) 매주 일 02:00  3) 직접(cron)
    백업 유지 개수           (7)
    복제 유저명              (replicator)
  [pgbackrest]
  Archive 사용               (Y)  WAL -> archive-push -> 저장소
    저장소 상위 경로         (/pgbackup) -> /pgbackup/pgbackrest
    stanza 이름              (main, HA 는 클러스터 이름)
  Backup 사용                (Y)
    백업 주기                1) 일요일 full + 월~토 diff (권장)  2) 매일 full  3) 직접
    full 보관 개수           (2)

진행
  rpm 설치 -> 소스 빌드 + contrib + extension(PGXS) [+ pgBackRest] -> ~/.postgresrc
  -> postgresql.conf 생성 -> initdb(--data-checksums) -> 기동
  -> 백업 스크립트 / 크론 -> pg_stat_statements, postgres 스키마
  pgbackrest 면 추가로: stanza-create -> check -> 첫 full 백업 -> 크론

기본 적용 설정
  shared_buffers = 메모리 1/4, effective_cache_size = 3/4, work_mem 자동 계산
  shared_preload_libraries = pg_stat_statements, track_io_timing = on
  log_min_duration_statement = 1000 (1초 이상 쿼리), log_statement = ddl
  lc_messages = en_US.utf8 (없으면 C)

완료 후
$ source ~/.bashrc
$ psql -c "select version();"
```

## 7. [3단계] HA 설치

```
etcd 서버에서 실행
$ bash postgresql_install.sh
  설치 모드: 2) ha

입력 항목 (순서대로)
  클러스터 이름              (pg-cluster)
  etcd 서버                  현재 서버 호스트명
  Primary / Standby          호스트명 (/etc/hosts 에 등록된 이름)
  SSH 포트                   (22)
  etcd 구성                  1) 단일 노드   2) 3노드 (권장)
  VIP 사용                   (N)  y 면 -> VIP 주소 / prefix(24) / 인터페이스(자동)
  watchdog                   1) automatic (권장)  2) required  3) off
  failsafe_mode              (Y)
  synchronous_mode           (N)  - 데이터 유실 0 이 필요하면 y (커밋 지연 증가)
  PostgreSQL Port            (5432)
  복제 유저명 / 패스워드
  Primary / Standby CPU, Memory
  엔진 / Data / Locale / 백업 선택 / Archive / Backup   (single 과 동일)
  [basebackup] 백업 서버     Primary 또는 Standby (기본: Standby)
```

```
  [pgbackrest]              저장소 = etcd 서버, 백업은 etcd 서버 크론이
                            Standby 에서 복사 (백업 서버 질문 없음)

확인 화면에서 etcd / VIP / 안전장치 / 백업 값 확인 후 Y

진행 단계
  pre-flight  SSH, sudo, /etc/hosts, VIP 도구(ip/arping/curl), VIP ping 점검
              하나라도 실패하면 조치 방법 출력 후 중단
  PHASE 1     etcd 설치 + systemd 등록 (1노드 / 3노드)
  PHASE 2     etcd 서버에서 PG + extension (+ pgBackRest) 빌드
  PHASE 3     Primary: 엔진 배포, rpm, .postgresrc, patroni.yml, patroni.service
  PHASE 4     Standby: 엔진(Primary -> Standby), 나머지 동일
  PHASE 5     VIP 스크립트 배포 + 보정 cron, watchdog 준비 (softdog, /dev/watchdog)
              Patroni 설치 -> Primary 기동 (REST /primary=200 대기)
              -> Standby 기동 + 자동 클론 (REST /replica=200 대기)
              -> postgresql.base.conf 에 튜닝값 적용, 백업 / WAL 도구 배포
              -> VIP 위치 확인, vip_watch 배포 + 1회 실행 결과 출력
  PHASE 6     etcd 서버 빌드 결과물 삭제
              (pgbackrest 면 etc/backup/pgbackrest 만 남김 = 저장소용)
              pgbackrest: DB 노드 -> etcd SSH 키 등록, 설정 배포,
              stanza-create -> check -> 첫 full 백업 -> 크론 (etcd 서버)

알아둘 것
  - archive_mode / archive_command 는 patroni.yml DCS 파라미터에 들어가 첫 기동부터 동작
  - shared_buffers 같은 재시작 항목은 patronictl restart 후 반영
    (postgresql.conf 는 Patroni 관리, 튜닝값은 postgresql.base.conf)
  - etcd 서버에 이미 single PostgreSQL (pgmon 저장소 DB) 이 같은 엔진 경로에
    있으면, PHASE 2 전에 /postgres -> /postgres.ha_keep_<시각> 로 옮겨두고
    (실행 중이면 중지) PHASE 6 에서 되돌린 뒤 다시 기동한다. 실패해도 되돌림.
  - 자동 failback 은 없음. 원래 노드로 돌리려면 switchover

완료 화면에 etcd / Primary / Standby / VIP / 감시자 / 백업 저장소가 표시됨

노드 추가 / DR : 설치 후 etcd 서버에서  bash add_node.sh   (운영가이드 5-4)

pgmon 저장소 DB : HA 설치 후 etcd 서버에서 single 로 한 번 더 설치
                  $ bash postgresql_install.sh  ->  1) single
                  $ bash monitoring/pgmon/pgmon.sh
```

## 8. 무인 설치 (응답 파일)

```
$ cp install.conf.sample install.conf && chmod 600 install.conf
$ vi install.conf
$ bash postgresql_install.sh -c install.conf      # 빠진 항목만 질문
$ bash postgresql_install.sh -c install.conf -y   # 완전 무인 (빠진 항목 = 기본값)

HA 예시 (값은 환경에 맞게)
  MODE_SEL=2
  CLUSTER_NAME=pg-cluster
  HA_ETCD_HOST=pgetcd
  HA_PRIMARY_HOST=pgdb1
  HA_STANDBY_HOST=pgdb2
  ETCD_MODE_SEL=2
  HA_VIP_USE=y
  HA_VIP=10.0.0.100
  HA_VIP_PREFIX=24
  HA_WATCHDOG_SEL=1
  HA_FAILSAFE=y
  HA_SYNC_MODE=n
  REPL_PASS_HA=<복제 패스워드>

pgBackRest 예시
  BACKUP_TOOL_SEL=pgbackrest
```

```
  USE_ARCHIVE=y
  USE_BACKUP=y
  PGBR_BASE=/pgbackup        # -> /pgbackup/pgbackrest
  PGBR_STANZA=pg-cluster     # single 기본 main
  PGBR_CRON_SEL=1            # 1=일요일 full + 월~토 diff
  PGBR_KEEP_FULL=2
나머지 키는 install.conf.sample 주석 참고
```

## 9. 설치 후 확인

```
[single]
 [ ] psql -c "select version();"
 [ ] psql -c "show shared_preload_libraries;"        -> pg_stat_statements
 [ ] psql -c "select * from pg_stat_archiver;"       -> failed_count = 0
 (basebackup)
 [ ] crontab -l                                      -> pg_basebackup, wal_cleanup
 [ ] bash /postgres/app/etc/backup/basebackup/pg_basebackup.sh   (1회 수동 백업)
 [ ] bash /postgres/app/etc/backup/pitr_restore.sh --list
 (pgbackrest)
 [ ] pgbackrest --stanza=main info                   -> status ok, full backup 1개
 [ ] crontab -l                                      -> pgbackrest full / diff

[HA]  (Primary 에서)
 [ ] patronictl -c /postgres/app/etc/patroni/patroni.yml list
       Leader 1개, Replica 1개, State=streaming, Lag 0
 [ ] patronictl -c /postgres/app/etc/patroni/patroni.yml show-config
       failsafe_mode / synchronous_mode / archive 값 확인
 [ ] etcdctl --endpoints=http://<etcd>:2379 endpoint health      (3노드면 3개)
 [ ] ip -o -4 addr show | grep pgvip         -> Primary 에만 VIP
 [ ] ss -lntp | grep 5432                    -> 0.0.0.0:5432 (VIP 접속 받기 위해)
 [ ] ls -l /dev/watchdog                     -> 실행 계정 소유 (watchdog 사용 시)
 [ ] journalctl -u patroni | grep -i watchdog
 [ ] crontab -l                              -> vip_check, wal_cleanup
                                                (basebackup 백업 서버는 pg_basebackup 도)
(etcd 서버에서)
 [ ] bash ~/pgvip/vip_watch.sh --once        -> "OK 리더=[primary] VIP보유=[primary]"
 [ ] crontab -l                              -> vip_watch
 [ ] pgbackrest --stanza=pg-cluster info     -> status ok, full backup 1개  (pgbackrest)
 [ ] pgbackrest --stanza=pg-cluster check    -> Primary archive-push 정상    (pgbackrest)
 [ ] crontab -l                              -> pgbackrest full / diff     (pgbackrest)
(앱 서버에서)
 [ ] psql -h <VIP> -c "select pg_is_in_recovery();"  -> f

[pgmon]
 [ ] bash monitoring/pgmon/pgmon_ctl.sh cron-install   (재부팅 자동 시작)
 [ ] bash monitoring/pgmon/pgmon_check.sh              -> 리포트 FAIL 없음

[보안 - 운영 전 필수]
 [ ] single: postgres 패스워드 설정, pg_hba.conf 의 local trust 조정
 [ ] HA: patroni.yml superuser 패스워드(기본 postgres) 변경, /24 trust 범위 검토
 [ ] pgmon role 패스워드(기본 = role 이름) 변경
```

## 10. 동작 테스트

### 10-1. 자동 - ha_test.sh (etcd 서버, 설치 계정)

```
  $ bash ha_test.sh                   T0 ~ T6
  $ bash ha_test.sh --only T3,T5      일부만
  $ bash ha_test.sh --with-watchdog   T7 포함 (노드가 실제로 재부팅됨)
  결과: ~/ha_test_report_<시각>.txt  (끝나면 시작 시점 Leader 로 되돌림)

  T0  사전 상태        Leader/Replica, VIP 위치, listen, watchdog
  T1  쓰기 / 복제      VIP 로 INSERT -> Replica 반영
```

```
T2  백업 / WAL 도구  pg_basebackup 1회, wal_cleanup --dry-run, pitr_restore --list
T3  switchover       전환 시간, VIP 이동, 쓰기 재개 시간 -> 되돌리기
T4  Patroni 정지     Leader patroni stop -> 자동 failover -> 재합류
T5  VIP 보정         VIP 강제 제거 -> vip_check 가 다시 붙이는지
T6  etcd 정지        failsafe_mode: etcd 가 죽어도 쓰기 계속
T7  watchdog 펜싱    Leader Patroni 를 멈춰 노드 리부트 확인
```

### 10-2. 수동 - 자동화하지 않은 것

```
준비: 터미널 3개
  (A) etcd 서버  : watch -n 2 "bash ~/pgvip/vip_watch.sh --once"
  (B) Primary    : tail -f /pgdata/log/vip.log
  (C) 앱 서버    :
        while true; do
          psql -h <VIP> -Atc "select pg_postmaster_start_time(), pg_is_in_recovery(), now()"
          sleep 1
        done
        (postmaster 기동 시각이 바뀌면 다른 노드로 붙은 것)

전원 OFF       Primary VM 강제 정지 / 전원 차단
               기대: 약 30~40초 후 Standby 승격 + VIP 이동
               복구: 서버 기동 -> Patroni 가 Replica 로 재합류
네트워크 단절  Primary 콘솔에서 서비스 인터페이스 down
               기대: Primary 스스로 demote + VIP 제거, Standby 승격 + VIP 추가
전체 재부팅    세 서버 순서 상관없이 재부팅
               기대: etcd -> Patroni 자동 기동, Leader 1개로 수렴, VIP 1개
               확인: patronictl list, vip_watch --once, pgmon_ctl.sh status
부하 중 전환   pgbench 로 부하를 건 채 switchover / failover
               pgbench -h <VIP> -i -s 10 postgres
               pgbench -h <VIP> -c 8 -T 300 postgres
```

### 10-3. pgmon 점검

```
$ bash monitoring/pgmon/pgmon_check.sh               C1~C6
$ bash monitoring/pgmon/pgmon_check.sh --alert-test  C7 Alert 시험 포함
```

## 11. 설치 문제 해결 (실서버에서 겪은 것 포함)

| 증상 | 원인 / 조치 |
|---|---|
| pre-flight SSH 실패 | 키 교환 방향 확인<br>ssh -o BatchMode=yes <호스트> 로 직접 테스트 |
| pre-flight sudo 실패 | /etc/sudoers.d/<계정> NOPASSWD, 권한 440 |
| /etc/hosts 항목 없음 | 세 서버 모두 같은 내용으로 등록 |
| PKG 에 rpm 이 몇 개뿐 | 다운로드 서버에 이미 설치된 패키지는 dnf 가 건너뜀<br>-> 최신 pg_download.sh 는 빠진 것을 개별 재다운로드<br>PKG 를 보충했으면 installer/PKG.tar.gz 삭제 후 재실행 |
| configure: gcc 없음 | PKG 에 gcc 가 없음 -> pg_download.sh 다시 실행 |
| configure 에서 옵션 빠짐 경고 | 해당 -devel rpm 이 PKG/ 에 없음 -> --verify 로 확인 |
| extension 빌드 실패 | 설치는 계속됨. 로그에서 실패 목록 확인 후 수동 빌드 |
| locale 오류로 initdb 실패 | locale -a 로 존재 확인, glibc-langpack-ko 설치 |
| Patroni 설치 실패 (pip3 없음) | PKG 에 python3-pip 필요 (manifest 에 포함됨) |
| patronictl 실행 시 ImportError | ydiff 휠 누락 -> pg_download.sh 다시 (휠 자동 변환) |
| Patroni 기동 실패 (psycopg2) | patroni_pkgs 에 psycopg2-binary 휠 필요 (자동 포함) |
| Patroni 기동 실패 (기타) | journalctl -u patroni -n 100<br>etcd 접속(2379), patroni.yml 들여쓰기 확인 |
| Standby 가 클론 안 됨 | Primary <-> Standby 5432 방화벽, 복제 유저 / pg_hba |
| VIP 로 접속 거부 | listen 이 노드 IP 만이면 VIP 접속 불가<br>-> 최신 버전은 0.0.0.0 (ss -lntp 로 확인) |
| VIP 가 안 붙음 (cron) | cron PATH 에 /usr/sbin 없음 -> 최신 스크립트는 PATH 지정<br>/pgdata/log/vip.log, sudo -n ip addr 동작 확인 |
| VIP 는 붙었는데 접속 안 됨 | 스위치 ARP 캐시 -> sudo arping -U -c 3 -I <dev> <VIP> |
| vip.log "Address already assigned" | 이미 붙은 VIP 에 add 를 다시 시도한 로그.<br>VIP 가 Primary 한 곳에만 있으면 무시해도 됨 |
| watchdog 경고 (사용 불가) | lsmod \| grep softdog, ls -l /dev/watchdog<br>다른 프로세스 점유 여부 (sudo lsof /dev/watchdog) |

| | |
|---|---|
| pgBackRest 빌드 실패 | meson / bzip2-devel / lz4-devel 확인 (PKG)<br>meson rpm 이 없으면 installer/pgbackrest/wheels 사용 |
| pgBackRest check 실패 (HA) | DB 노드 -> etcd 서버 SSH:<br>ssh <DB노드> "ssh -o BatchMode=yes <etcd> true"<br>DB 노드 /postgres/app/etc/backup/pgbackrest/log/ |
| 설치 로그 | /pgdata/log/pg_install.log<br>HA 는 /pgdata/log/ha_setup.log 도 |

## 12. 다시 설치할 때 (정리 방법)

설치가 중간에 실패해서 처음부터 다시 할 때. 설치 스크립트는 DB 노드의 데이터를
지우지 않으므로 아래를 먼저 정리한다 (운영 데이터가 있으면 절대 하지 말 것).

```
[DB 노드 - Primary / Standby 각각]
  sudo systemctl stop patroni; sudo systemctl disable patroni
  sudo rm -f /etc/systemd/system/patroni.service; sudo systemctl daemon-reload
  rm -rf /pgdata/data /postgres/app
  crontab -l | grep -v -e vip_check -e wal_cleanup -e pg_basebackup | crontab -
  (VIP 가 남아 있으면) sudo ip addr del <VIP>/<prefix> dev <dev>
  (3노드 etcd 였으면) sudo systemctl stop etcd; sudo rm -rf /var/lib/etcd

[etcd 서버]
  sudo systemctl stop etcd          (재설치 시 /var/lib/etcd 는 스크립트가 지움)
  crontab -l | grep -v -e vip_watch -e pgbackrest | crontab -
  (pgBackRest 저장소를 새로 만들려면) rm -rf /pgbackup/pgbackrest

그 다음 최신 패키지로 다시:  bash postgresql_install.sh
```

## END